CONTRACT NOTE REWORK

Contract Note Data Source Extensibility

Technical Walkthrough

D55 Consulting

July 2026

BrytEnergy

Background

Contract notes today can only show data that arrives in the contract payload. If the business wants to include something new, a credit score, a loyalty tier, a bespoke figure from another system, that has historically meant a developer change to the render pipeline.

This capability removes that dependency. Business users manage data sources in SageMaker Unified Studio; once a source is subscribed to the contract note project, it becomes automatically discoverable in the Admin Portal. Users attach it to a template, use its fields in the section designer, and at render time the pipeline fetches the matching data and merges it in, all without a code change or redeployment.

The join key throughout is the BrytNumber (the customer reference already present in the contract payload). Every data source must expose a bryt_number column so it can be matched to the customer being rendered.

> ### What changes, in one line
>
> The template management experience and the render pipeline stay the same. What is added is a self-service path for business users to bring in new customer data: subscribe a source in Unified Studio, attach it to a template, use its fields, and have the pipeline enrich the contract at render time.

How it works

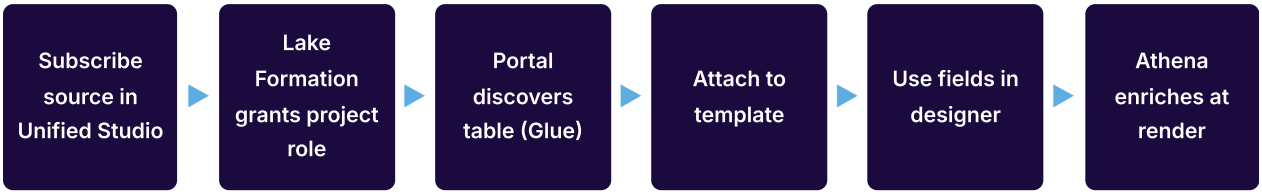
The capability spans three touch points: discovery, design, and render.

Discovery. When a user subscribes a data source in Unified Studio, Lake Formation grants land on the project role. The Admin Portal, assuming that same role, lists the newly available tables from the Glue Data Catalog, showing only those with a bryt_number column so they can actually be joined.

Design. A user attaches a data source to a template, and its columns then appear as fields in the section editor, grouped and namespaced by source, alongside the core contract fields. Shared sections automatically track which data sources they depend on.

Render. When the pipeline renders a template, it looks up the attached data sources, queries each one via Athena using the BrytNumber, and merges the results into the contract data before the sections are rendered.

From subscription to rendered field

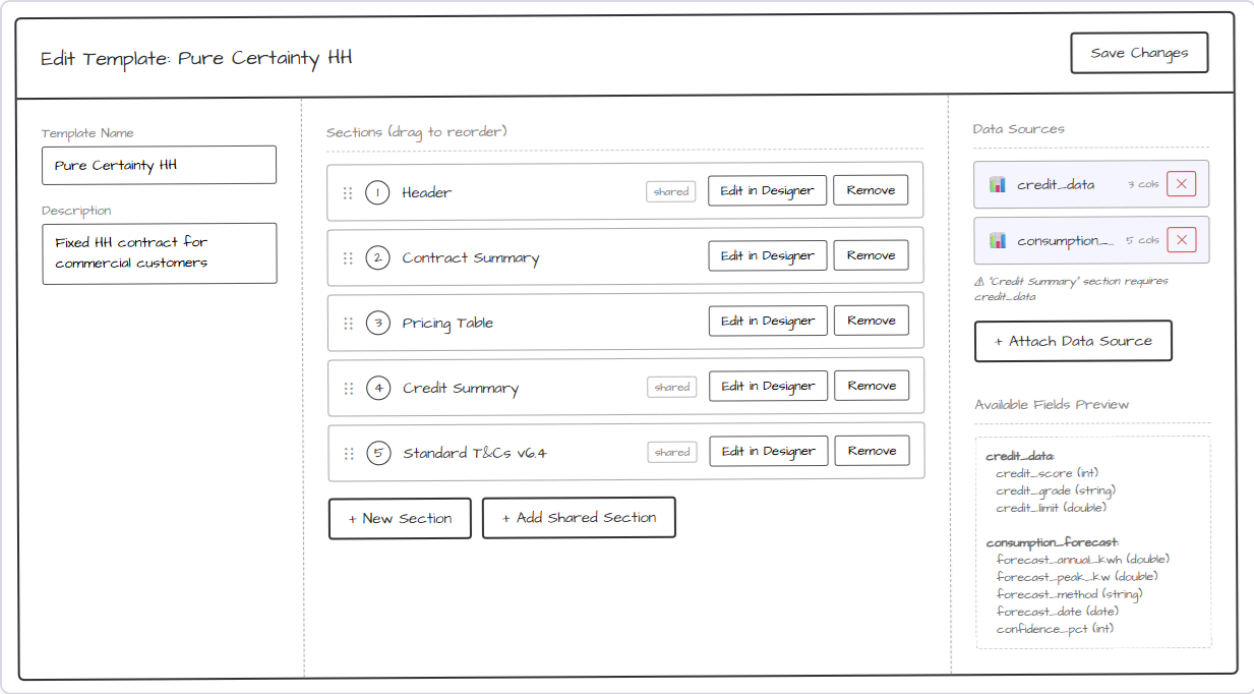


A newly subscribed data source is usable end-to-end with no code change or redeployment.

The screen

This capability extends the existing Template Edit screen rather than adding new screens. A data sources panel is added, and the section editor gains a data source field group.

Template Edit - Data Sources panel



The Template Edit screen gains a Data Sources panel showing which sources are attached to the template. From here a user attaches an available source or detaches one they no longer need.

KEY INTERACTIONS

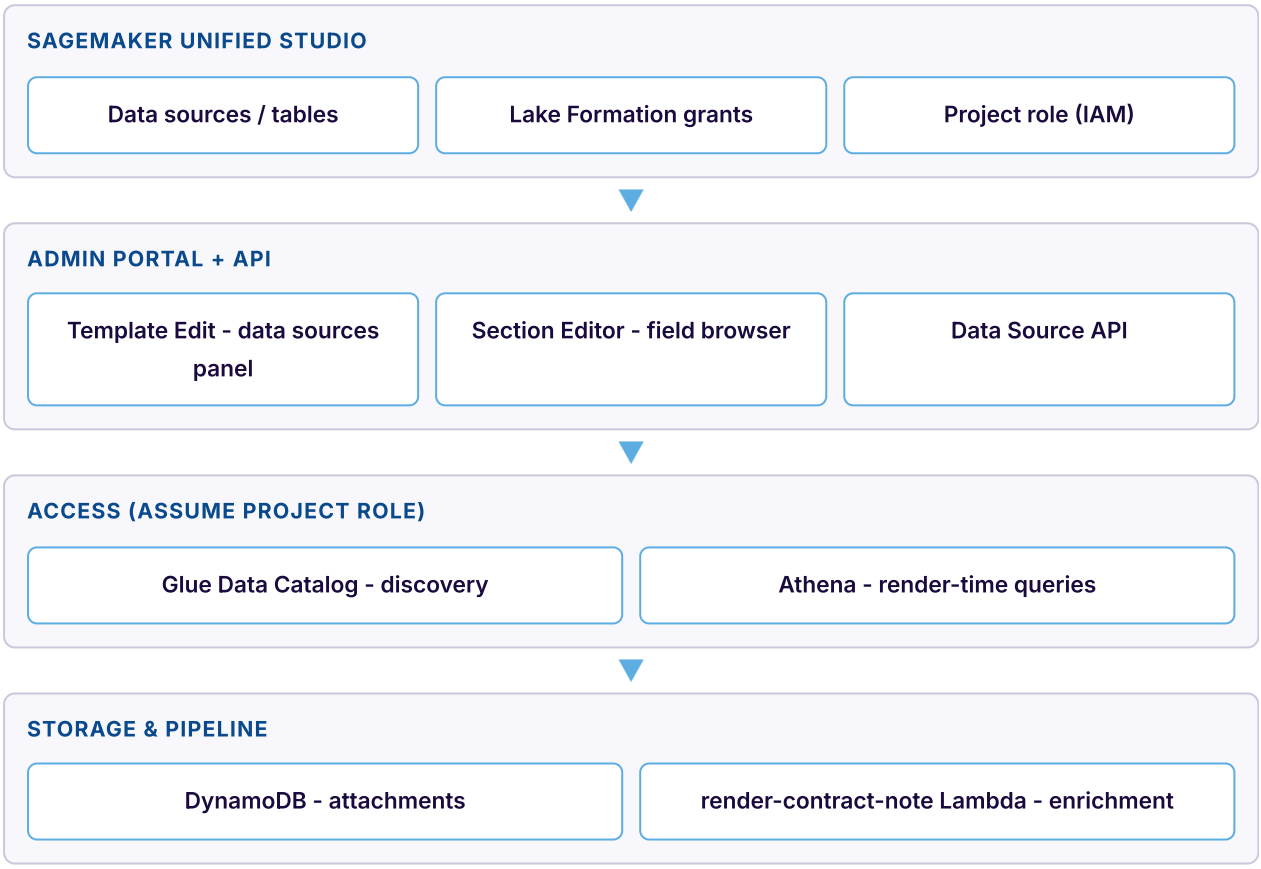
- Attach Data Source opens a picker of available (unattached) sources
- Each attached source shows its name and column count
- Detach warns and lists affected sections if fields are in use
- Attached source columns become available in the section editor

BEHIND THE SCREEN

- Attachments are stored as records under the template in the existing DynamoDB table
- The picker is populated from the Glue catalog via the project role
- Only tables with a bryt_number column are offered
- Shared sections separately track their own data source dependencies

System architecture

The Admin Portal and render pipeline both reach data sources by assuming the Unified Studio project role, which already holds the Lake Formation grants. Discovery goes through the Glue Data Catalog; render-time enrichment goes through Athena. Attachment records live in the same DynamoDB table used for template management.



Access to data sources is inherited via the project role, so new subscriptions need no IAM changes.

Key design decisions

The design leans on the existing Unified Studio governance rather than reinventing access control.

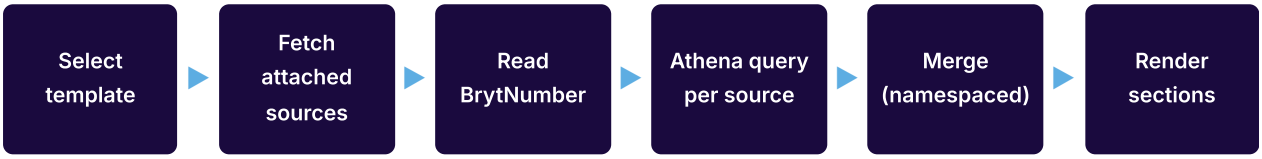
DECISION	CHOICE	WHY
Data source access	Assume the Unified Studio project role	Inherits all Lake Formation grants automatically; no per-table IAM configuration
Catalogue discovery	Glue Data Catalog API	The standard AWS metadata layer; Unified Studio uses Glue underneath

DECISION	CHOICE	WHY
Render-time query	Athena	Serverless SQL over Glue/Iceberg tables; handles varied storage formats
Field namespacing	{source}.{column}	Avoids collisions between data sources and core contract fields; makes provenance clear
Template binding	Explicit attachment per template	Limits render-time queries to what's actually needed; keeps dependencies visible
Join constraint	Table must have a bryt_number column	Enforces join-ability and filters out tables that can't be matched to a customer
Section dependencies	Auto-tracked from field references	No manual config; derived by scanning which data source fields a section uses

Render-time enrichment

Enrichment slots into the existing render pipeline as a single new step between template selection and section rendering. Once a template is chosen, the pipeline gathers its attached data sources, reads the BrytNumber from the contract data, and queries each source in parallel via Athena. The results are merged into the contract data under a per-source namespace, then rendering proceeds exactly as before.

Enrichment step in the pipeline



Queries run in parallel to keep added latency low.

How enrichment handles the edge cases

SITUATION	BEHAVIOUR
Row found for the BrytNumber	Columns merged in under {table}.{column} and available to fields
No row for the BrytNumber	Warning logged; rendering continues with those fields empty
Multiple rows returned	First row used; a warning is logged
Athena query fails or times out	Error logged to the error bucket; rendering halts (no partial PDF)

What we store

No new table is needed. Two record types are added to the existing ContractNoteTemplates table.

RECORD	HOLDS
Template data source	The database + table attached to a template, display name, and who attached it / when

RECORD	HOLDS
Shared section dependency	The data sources a shared section requires, derived from the fields it uses
Field reference (in schema)	Data source fields are stored namespaced, e.g. credit_data.credit_score

Delivery breakdown

The build is grouped into the following areas.

AREA	SCOPE
Infrastructure & IAM	Project role trust policy for Lambda assumption, Athena workgroup + results bucket, DynamoDB record types
Glue discovery client	Assume project role, list tables, filter to bryt_number tables, fetch column detail
Data Source API	List available, get columns, attach, detach (with field-in-use check), list attached
Render enrichment	Athena query executor, enrichment orchestrator, integration into the render pipeline
Template Edit panel	Data sources panel, attach/detach with warnings, data source picker dialog
Section editor fields	Data source field group, dependency tracking, missing-dependency prompt, dependency display
Integration	CDK wiring for routes, role assumption, Athena, and end-to-end validation

A note on cost

Athena is billed by data scanned. For large data sources it is worth partitioning or pruning on bryt_number so each render-time lookup scans as little as possible. Queries also run in parallel across sources to keep the added render latency down.

Testing note

The build is designed around 10 correctness properties (bryt_number-only discovery, attachment round-trips, namespaced enrichment, graceful handling of missing rows, and fail-safe on query errors). Property-based and integration tests are marked optional in the plan and can be deferred for a faster MVP, but they are recommended given enrichment feeds live customer documents.